

Discuss the role of indexes in a database. Describe what they are, what should be indexed and how you know if you need one. Talk about the different types of indexes and when they should be used. Talk about the storage needs for an index as it relates to planning and how clustering may or may not negate the need for an index

Definition of Index:

It's a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space to maintain the index data structure.

The role of indexes:

According to the definition above, it's a data structure that speeds up the retrieval operations, but it comes with the cost like additional writes and storage space.

What indexes are:

An index is a copy of selected columns of data, from a table, that is designed to enable very efficient search. An index normally includes a "key" or direct link to the original row of data from which it was copied, to allow the complete row to be retrieved efficiently.

When would I need indexes:

When I want to speed up the data retrieval.

Types of Indexes:

There are many types of indexes, including clustered index, non-clustered index, column store index, filtered index, hash index, unique index etc.,.

Clustered index:

It sorts and store rows data in a table or view based on their fundamental values.

Clustered indexes do not have to be explicitly declared.

Created when the table is created.

Use the primary key sorted in ascending order.

Non-clustered index:

Non-clustered indexes are sorted references for a specific field, from the main table, that hold pointers back to the original entries of the table. Non-clustered indexes point to memory addresses instead of storing data themselves. This makes them slower to query than clustered indexes but typically much faster than a non-indexed column.

Unique index:

A unique index in SQL ensures that all values in the indexed column (or columns) are distinct. This means it prevents duplicate entries from being added to the index, making sure that every value is unique.

Filtered index:

A filtered index is a type of index in SQL that is created for a specific subset of rows in a table, rather than the entire table. It applies a filter condition, allowing you to index only the rows that meet certain criteria (e.g., rows with specific values in a designated column). This is particularly useful when a column contains many irrelevant values and only a few relevant ones are frequently queried.

Column store index:

A column store index is a type of index designed for efficiently storing and querying large data sets, particularly in data warehousing environments. Unlike traditional row-oriented storage, which organizes data by rows, a column store index organizes data by columns. This structure is particularly beneficial for workloads that involve large volumes of data and complex queries

Hash Index:

A hash index in SQL is a data structure that allows for fast lookups of records based on a specific key. It functions by using a hash function, denoted as $F(K, N)$, where K is the key you are searching for, and N represents the number of buckets (or slots) in the index

Storage needs for index:

1. Determine the size of the source structure
2. Determine the size of the target index structures
3. Determine the additional temporary disk space for sorting

How clustering may or may not negate the need for an index

A cluster provides an optional method of storing table data. A cluster is made up of a group of tables that share the same data blocks. The tables are grouped together because they share common columns and are often used together. So, when a query accesses data, it can often retrieve it in fewer I/O operations, reducing the need for separate indexing.

https://en.wikipedia.org/wiki/Database_index

<https://www.atlassian.com/data/sql/how-indexing-works>

<https://www.besanttechnologies.com/sql-indexes>

<https://learn.microsoft.com/en-us/sql/relational-databases/indexes/index-disk-space-example?view=sql-server-ver16>

<https://docs.oracle.com/en/database/oracle/oracle-database/21/admin/managing-clusters.html>